claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-adc6ab0bc6fd48efa.jsonl`
- SHA-256: `9adc123e8f0d6fbf9c9dbaeac4a9dceadd6b10587d6140ea7af2c44fed088599`
- Source modified: `2026-06-21T13:55:07+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:52:09.113Z)

Research how local-AI apps handle torch GPU vs CPU vs Apple MPS at INSTALL/RUN time across platforms, given torch CUDA wheels require --index-url. Options: install-time hardware detection + correct --index-url; an installer/launcher that picks the wheel; CPU-default with optional GPU upgrade; documented extras. How do Ollama/ComfyUI/Faster-Whisper handle this? Map to our DeviceContext (cuda/auto/cpu) + the gpu/cpu pyproject extras. Use web search.

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be

MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW
packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a
"self-hosted webserver + UI the user spins up" (NOT a desktop app).
khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T13:52:12.400Z)

I'll research how local-AI apps handle torch GPU/CPU/MPS at install/run time. Let me start by
grounding in our actual code, then do web research.

3. user (2026-06-21T13:52:13.717Z)

```
1       # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2       #
3       # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4       # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5       # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6       # auto-detection would otherwise miss the un-__init__'d `backend` root.
7       #
8       # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9       # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10      # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11      # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12      [build-system]
13      requires = ["hatchling"]
14      build-backend = "hatchling.build"
15
16      [project]
17      name = "badminton-highlight-indexer"
18      version = "0.1.0"
19      description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20      readme = "README.md"
21      requires-python = ">=3.10"
22      license = { text = "MIT" }
23      authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24      keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26      # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27      dependencies = [
28          "fastapi>=0.111.0",
29          "uvicorn>=0.30.1",
30          "opencv-python>=4.9.0.80",
31          "numpy>=1.26.4",
32          "pydantic>=2.7.1",
33          "jinja2>=3.1.4",
34          "python-dotenv>=1.0.0",
35          "google-genai>=2.4.0",
36          "supervision>=0.21.0,<0.30.0",
37      ]
38
39      [project.optional-dependencies]
40      # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41      dev = [
42          "pytest",
43          "pytest-cov",
44          "httpx",        # fastapi TestClient transport
45          "ruff",
46          "mypy",
47      ]
48
49      # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50      # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
```

```
51      # fetched over SSH; if you lack access, simply omit this group ? reporting
52      # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53      #   pip install -e ../sports-obsreport --no-build-isolation
54      reporting = [
55          "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56      ]
57
58      # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59      # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60      # express, so they are NOT declarable here. Install them out-of-band instead:
61      #   # CUDA 12.4 box:
62      #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63      #   # CPU-only box (also what CI uses):
64      #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65      gpu = []
66
67      # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68      # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69      # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70      # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71      # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72      storage-s3 = ["boto3>=1.34"]
73      storage-gcs = ["google-cloud-storage>=2.16"]
74
75      [project.scripts]
76      # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77      indexer = "backend.main:main"
78
79      [tool.hatch.metadata]
80      # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81      # reference*); hatchling requires opting in to allow those in metadata.
82      allow-direct-references = true
83
84      [tool.hatch.build.targets.wheel]
85      # `backend` is a namespace package (no __init__.py); list both import roots
86      # explicitly so an editable/wheel install exposes `import backend` and
87      # `import training` regardless of cwd.
88      packages = ["backend", "training"]
89
```

4. assistant (2026-06-21T13:52:17.038Z)

I have the pyproject. Now let me read the key code files that define our DeviceContext, profiles, and the worker/main entrypoints, while kicking off web research in parallel.

5. user (2026-06-21T13:52:19.133Z)

```
Found 32 files
backend\infrastructure\database.py
backend\config\models.py
backend\worker\__main__.py
backend\config\startup.py
backend\config\__init__.py
backend\config\presets.py
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\detectors\device.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\detectors\wasb_infer.py
```

backend\pipeline\stitcher\compiler.py
backend\pipeline\segmenters\motion.py
backend\eval\golden_fixtures.py
backend\eval\experiment.py
backend\pipeline\detectors\tracknet_runner.py
backend\eval\segmentation_metrics.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\utils\run_telemetry.py
backend\pipeline\detectors\person_detector.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\pipeline\validators\pts_continuity.py
backend\eval\annotations.py
backend\eval\regression.py
backend\api\static\index.html
backend\sports\badminton.py
backend\eval__init__.py

6. user (2026-06-21T13:52:26.138Z)

Web search results for query: "ComfyUI install torch CUDA index-url GPU CPU Apple MPS launcher"

Links: [{"title":"System Requirements - ComfyUI Official Documentation","url":"https://docs.comfy.org/installation/system_requirements"},{"title":"How to install ComfyUI - Stable Diffusion Art","url":"https://stable-diffusion-art.com/how-to-install-comfyui/"},{"title":"Torch not compiled with CUDA enabled on Mac Pro M3 running Janus Image Generation · Issue #6783 · Comfy-Org/ComfyUI","url":"https://github.com/comfyanonymous/ComfyUI/issues/6783"},{"title":"GitHub - Comfy-Org/ComfyUI: The most powerful and modular diffusion model GUI, api and backend with a graph/nodes interface. · GitHub","url":"https://github.com/Comfy-Org/ComfyUI"},{"title":"ComfyUI Recommended PyTorch Version 2026: Quick Guide","url":"https://smartart.live/articles/277-comfyui-recommended-pytorch-version-2026.html"}]

Based on the search results, here's information about installing PyTorch with ComfyUI across different platforms:

NVIDIA GPU (CUDA)

For NVIDIA GPUs, install stable PyTorch with CUDA 13.0 using: `pip install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu130`

Apple Silicon (MPS)

macOS 12.3 or higher is required for MPS acceleration support. For Apple Mac silicon (M1, M2, M3, or M4), install PyTorch nightly, following the Accelerated PyTorch training on Mac Apple Developer guide.

CPU-Only Installation

For CPU-only installation, use the `--cpu` parameter (though it will be slower).

Troubleshooting

If you encounter a "Torch not compiled with CUDA enabled" error, it usually means you installed a CPU-only PyTorch build by accident; fix it by reinstalling from the correct CUDA wheel index (cu130 for NVIDIA).

Platform Support Summary

ComfyUI supports NVIDIA GPU, AMD GPU (Windows/Linux, RDNA 3/3.5/4), Intel Arc GPU, Apple Silicon with Metal acceleration, and CPU-only mode.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

7. user (2026-06-21T13:52:26.190Z)

Web search results for query: "how Ollama installer detects GPU CUDA vs CPU at install time"

Links: [{"title":"How to Install Ollama with NVIDIA GPU Support (CUDA Guide) | Markaicode","url":"https://markaicode.com/install-ollama-nvidia-gpu-cuda-support/"},{"title":"Ollama GPU Not Detected: How to Fix CUDA and ROCm Errors -","url":"https://www.serverman.co.uk/server/ollama-gpu-not-detected-how-to-fix-cuda-and-rocm-errors/"},{"title":"Ollama Not Using GPU? Fix CPU-Only Inference on Windows, WSL2, and Linux (2026)","url":"https://runaihome.com/blog/ollama-not-using-gpu-fix-cpu-2026/"},{"title":"Fix Ollama GPU Detection Issues: Complete Driver Configuration Guide | Markaicode","url":"https://markaicode.com/fix-ollama-gpu-detection-driver-configuration/"},{"title":"Ollama Not Using GPU: Complete Fix Guide (2026) | InsiderLLM","url":"https://insiderllm.com/guides/ollama-not-using-gpu-fix/"},{"title":"CUDA Detection issues · Issue #12136 · ollama/ollama","url":"https://github.com/ollama/ollama/issues/12136"},{"title":"Ollama GPU Acceleration Configuration: CUDA, ROCm, and Metal Platform Guide · BetterLink Blog","url":"https://eastondev.com/blog/en/posts/ai/20260516-ollama-gpu-acceleration/"},{"title":"Hardware support - Ollama","url":"https://docs.ollama.com/gpu"},{"title":"Ollama Ubuntu 24.04 NVIDIA Install: Driver Pitfalls Guide | ITECS","url":"https://itecsonline.com/post/ollama-ubuntu-nvidia"}]

Based on the search results, here's how the Ollama installer detects GPU (CUDA) vs CPU at install time:

Automatic GPU Detection

The Ollama installer script automatically detects your GPU and configures Ollama accordingly, with no additional flags required. Ollama automatically detects NVIDIA GPUs when properly configured, eliminating the need for manual CUDA path configuration or separate CUDA Toolkit installations.

Detection Mechanism

Ollama automatically detects CUDA after installation with no extra configuration needed, just ensuring the driver is working. The detection happens at startup rather than strictly at install time?Ollama is designed to detect your GPU automatically at startup, and when it does, model inference is offloaded to the GPU.

Silent Fallback Behavior

Importantly, Ollama automatically falls back to CPU when it cannot detect a compatible GPU or the required compute framework (CUDA for NVIDIA, ROCm for AMD) is missing or misconfigured, and this happens silently with no prominent warning. Ollama silently falls back to CPU mode without obvious error messages when driver issues exist.

Key Requirements for Detection

For successful GPU detection, Ollama relies on specific runtime libraries to hand off compute to the GPU, and when those components are missing, mismatched, or misconfigured, it silently falls back to CPU. The main dependencies are the NVIDIA driver (not a separate CUDA Toolkit) and proper environment configuration.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

8. assistant (2026-06-21T13:52:30.047Z)

Let me read our device.py (the DeviceContext) and the native WASB runner, plus continue web research on Faster-Whisper and ComfyUI's portable launcher.

9. user (2026-06-21T13:52:30.586Z)

```
"""DeviceContext ? resolve the effective torch device for a detector runner.

COMPUTE_DECOUPLED_SERVING M4 (realizes the ``DeviceContext`` designed in the archived
``07-COMPUTE-ABSTRACTION``). A runner asks for a device by name; this turns the *requested*
device + a probed CUDA availability into the *effective* device to actually run on, and says
whether a CPU fallback happened.

The portability gap it closes: the native WASB runner hardcoded ``device='cuda'`` and its
healthcheck hard-failed on a box with no GPU, so a CPU-only Linux box (cpu-serving / a
no-GPU truly-local install / CI) could not run at all.

**Safety contract (so "cuda default unchanged" holds):**
  * ``"auto"``  ? ``cuda`` if a GPU is available, else ``cpu`` (the fallback). This is opt-in.
  * ``"cuda"``  ? honored verbatim; an *explicit* cuda request on a GPU-less box still hard-fails
    elsewhere (no silent slow-CPU downgrade of a config that asked for the GPU).
  * ``"cpu"`` / ``"mps"`` ? honored verbatim.
"""

from __future__ import annotations

from dataclasses import dataclass

AUTO = "auto"
CUDA = "cuda"
MPS = "mps"
CPU = "cpu"

# The device names a runner accepts. ``auto`` resolves; the rest are torch devices passed verbatim.
VALID_DEVICES = (AUTO, CUDA, MPS, CPU)


@dataclass(frozen=True)
class DeviceContext:
    """The requested device + a probed CUDA availability ? the effective device."""

    requested: str          # what config/env asked for: auto | cuda | mps | cpu
    cuda_available: bool     # probed in the runner's own torch env (not the main process)

    @property
    def effective(self) -> str:
        """The device the runner should actually use. ``auto`` resolves to cuda-if-available-else-cpu;
        any explicit device is returned verbatim."""
        if self.requested == AUTO:
            return CUDA if self.cuda_available else CPU
        return self.requested

    @property
    def fell_back(self) -> bool:
        """True when ``auto`` resolved to CPU because no GPU was found (worth a loud warning)."""
        return self.requested == AUTO and not self.cuda_available

    @property
    def cuda_required_but_missing(self) -> bool:
        """True when an EXPLICIT ``cuda`` request can't be satisfied ? the caller should hard-fail
        rather than silently downgrade (preserving the pre-M4 strict behaviour for
```

```
``device='cuda'``)."""
56              return self.requested == CUDA and not self.cuda_available
57
```

10. user (2026-06-21T13:52:37.857Z)

```
Web search results for query: "faster-whisper install torch CUDA CPU GPU CTranslate2 device auto
detection"

Links: [{"title":"GitHub - SYSTRAN/faster-whisper: Faster Whisper transcription with CTranslate2
· GitHub","url":"https://github.com/SYSTRAN/faster-whisper"},{"title":"CUDA compatibility with
CTranslate2 · Issue #1086 · SYSTRAN/faster-whisper","url":"https://github.com/SYSTRAN/faster-
whisper/issues/1086"},{"title":"Installing faster-whisper with GPU support via CTranslate2
(dependencies: CUDA>=11.2 CuDNN 8.x and CuBLAS) · GitHub","url":"https://gist.github.com/lmmx/49
c00238d3f05c6ec3a4afd659b9d01a/61da7b32a54d7657433d3ed4d97f3f561cf901cf"},{"title":"pytorch-
rocm-5.7.1 and ctranslate2 gfx803 - python detects cuda device / Kernel & Hardware / Arch Linux
Forums","url":"https://bbs.archlinux.org/viewtopic.php?id=297699"},{"title":"whisper-ctranslate2
· PyPI","url":"https://pypi.org/project/whisper-ctranslate2/"},{"title":"GitHub -
ThePlasmak/faster-whisper: An OpenClaw skill that uses faster-whisper (a faster implementation
of the Whisper transcription model) to transcribe audio, with additional features such as
speaker diarization · GitHub","url":"https://github.com/theplasmak/faster-
whisper"},{"title":"Faster Whisper transcription with
CTranslate2","url":"https://pypi.org/project/faster-whisper/"},{"title":"faster-whisper ·
PyPI","url":"https://pypi.org/project/faster-whisper/0.3.0/"},{"title":"insanely-fast-whisper ·
PyPI","url":"https://pypi.org/project/insanely-fast-whisper/"},{"title":"GitHub -
AIXerum/faster-whisper: faster-whisper is a reimplementation of OpenAI's Whisper model using
CTranslate2, which is a fast inference engine for Transformer models. This implementation is up
to 4 times faster than openai/whisper for the same accuracy while using less memory. The
efficiency can be further improved with 8-bit quantization on both CPU and GPU. ·
GitHub","url":"https://github.com/AIXerum/faster-whisper"}]

Here are the search results for faster-whisper installation with information about Torch, CUDA,
CPU/GPU, CTranslate2, and device auto-detection:
```

Key Information:

```
**Device Auto-Detection:**
By default the best hardware available is selected for inference. You can use the options
--device and --device_index to control manually the selection.

**Installation Overview:**
faster-whisper is a reimplementation of OpenAI's Whisper model using CTranslate2, which is a
fast inference engine for Transformer models. This implementation is up to 4 times faster than
openai/whisper for the same accuracy while using less memory. The efficiency can be further
improved with 8-bit quantization on both CPU and GPU.

**GPU Requirements:**
GPU execution requires the NVIDIA libraries cuBLAS 11.x to be installed on the system.

**CUDA Compatibility Issues:**
The latest versions of ctranslate2 only support CUDA 12 and cuDNN 9. For CUDA 11 and cuDNN 8,
the current workaround is downgrading to the 3.24.0 version of ctranslate2, for CUDA 12 and
cuDNN 8, downgrade to the 4.4.0 version of ctranslate2.

**PyTorch Installation:**
For CUDA 12.1 compatibility, you can install PyTorch with: `pip install torch --index-url
https://download.pytorch.org/whl/cu121`

The search results provide detailed information about compatibility requirements and
configuration options for running faster-whisper on both CPU and GPU systems.


REMINDER: You MUST include the sources above in your response to the user using markdown
hyperlinks.
```

11. user (2026-06-21T13:52:39.007Z)

Web search results for query: "ComfyUI portable windows embedded python bundles torch which wheel CUDA prebuilt"

Links: [{"title":"System Requirements - ComfyUI Official Documentation","url":"https://docs.comfy.org/installation/system_requirements"},{"title":"Install ComfyUI + SageAttention 2.2 on Windows 10/11 | RTX 3000/4000/5000 Guide 2026","url":"https://smartart.live/articles/machine-learning/284-install-comfyui-sageattention-22-on-windows-1011-ultimate-rtx-300040005000-guide-pytorch-cuda-triton-wheel-picker-dll-fix-2026.html"},{"title":"Pixal3D-ComfyUI/docs/windows_wheels.md at main · Saganaki22/Pixal3D-ComfyUI","url":"https://github.com/Saganaki22/Pixal3D-ComfyUI/blob/main/docs/windows_wheels.md"},{"title":"Releases · YanWenKun/ComfyUI-Windows-Portable","url":"https://github.com/YanWenKun/ComfyUI-Windows-Portable/releases"},{"title":"Setting up ComfyUI with Python 3.13.11, Torch 2.10, CUDA 13.0, and RTX 50 Series: Getting Trellis 2 Running??????@????","url":"https://note.com/kokonoe_kazari/n/nd10c723f277c?hl=en"},{"title":"How to Install SageAttention in ComfyUI (Windows) - Earngenix","url":"https://www.earngenix.com/tutorials/install-sageattention-comfyui"},{"title":"ComfyUI Installation Tutorial ? Creepybits","url":"https://zanno.se/comfyui-installation-tutorial/"}]

Based on the search results, here's information about ComfyUI portable Windows installations with embedded Python bundles and PyTorch wheels:

Windows Portable Build Configuration

The Windows portable build currently comes with Python 3.13 and PyTorch CUDA 13.0.

Embedded Python Structure

Portable uses embedded Python in a folder. For portable ComfyUI, use python_embeded\python.exe instead of venv\Scripts\python.exe. Additionally, after extracting the include/libs package, your python_embeded folder should contain include\ and libs\ alongside the existing Lib\ folder.

PyTorch Wheel Installation

When installing PyTorch wheels in portable ComfyUI installations, you need to match several parameters:

- Look at the filename (torch2.7, torch2.8, torch2.9, etc.). The patch version can differ, but minor version must match.
- Match CUDA line: CUDA 12 vs CUDA 13 matters. Use cu128 for CUDA 12.8 line, cu130 for CUDA 13.0 line.
- Many newer wheels use ABI3 (cp39-abi3), which works across Python ? 3.9, making Python version less critical.

Prebuilt Wheels Availability

For NVIDIA GPU - Install stable pytorch with CUDA 13.0: pip install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu130

A .whl file is a prebuilt Python package.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. assistant (2026-06-21T13:52:42.772Z)

Now let me read how DeviceContext is wired into the WASB runner and presets/startup, since the "two-torch-env" problem is the crux. Let me also research the ComfyUI desktop installer (the newer Electron app) and the uv/PyTorch index-pinning approach which is the modern best practice.

13. user (2026-06-21T13:52:44.207Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8       see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14          (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15          strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16        * weights / repo / python / scratch come from the **environment** first
17          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28      candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29      ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32      artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34      not the numbers.
35      """
36      import os
37      import shutil
38      import logging
39      import subprocess
40      from dataclasses import dataclass
41      from typing import Any, Dict, List, Optional, Tuple
42
43      from backend.pipeline.detectors.base import DetectorRunner
44      from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
45      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
46      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
47
48      logger = logging.getLogger(__name__)
```

```
49
50      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
51      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
52      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
53
54      # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
55      # so the "cuda True/False" string they parse can never drift apart.
56      _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
57
58
59      @dataclass
60      class NativeWasbConfig:
61          """Resolved settings for a Linux-native WASB run.
62
63          Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
64          over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
65          defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
66          """
67          repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
68          python_bin: str = "python"             # the `wasb` conda env python (invoked by
path, no activate)
69          weights_path: str = ""                 # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
70          sport: str = "badminton"
71          device: str = "cuda"                   # auto | cuda | mps | cpu (auto = cuda-if-
available-else-cpu)
72          scratch_dir: str = ""                  # frame-extraction scratch (env); "" = next
to the output dir
73          timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no
timeout)
74          keep_frames: bool = False              # retain the decoded frame cache after
success (debug only)
75          # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
76          # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
77          # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
78          stream_video: bool = True
79
80          @classmethod
81          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
82              w = (idx_cfg or {}).get("wasb", {}) or {}
83              env = os.environ.get
84              # stream_video is tri-state in config: None/absent => native default (stream);
an
85              # explicit True/False still wins (e.g. False for a container cv2 can't seek).
86              sv = w.get("stream_video")
87              return cls(
88                  repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
89                  python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
90                  weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
91                  sport=w.get("sport", "badminton"),
92                  device=env("WASB_DEVICE") or w.get("device", "cuda"),
93                  scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
"",
94                  timeout_sec=int(w.get("timeout_sec", 1800)),
95                  keep_frames=bool(w.get("keep_frames", False)),
96                  stream_video=(True if sv is None else bool(sv)),
97              )
98
```

```
99
100     class NativeWasbRunner(DetectorRunner):
101         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
102
103         def __init__(self, cfg: NativeWasbConfig):
104             self.cfg = cfg
105             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
106             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
107             self._cuda_available: Optional[bool] = None
108
109         # --- low-level native exec (the single place tests patch)
-------------------------
110         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
111             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
112             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
113                                   timeout=(self.cfg.timeout_sec or None))
114
115         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) ------
116         def _probe_cuda(self) -> bool:
117             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
118             (missing python / timeout / import error) is treated as 'no CUDA'."""
119             try:
120                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
121             except (FileNotFoundError, subprocess.TimeoutExpired):
122                 return False
123             return res.returncode == 0 and "cuda True" in (res.stdout or "")
124
125         def _cuda_is_available(self) -> bool:
126             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
127             if self._cuda_available is None:
128                 self._cuda_available = self._probe_cuda()
129             return self._cuda_available
130
131         def _effective_device(self) -> str:
132             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
133             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
134             if self.cfg.device == AUTO:
135                 return DeviceContext(AUTO, self._cuda_is_available()).effective
136             return self.cfg.device
137
138         def _repo_src(self) -> str:
139             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
140
141         def _copy_infer_script(self) -> bool:
142             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
143             repo_src = self._repo_src()
144             try:
145                 os.makedirs(repo_src, exist_ok=True)
146                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
147                 return True
148             except OSError as e:
149                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
150                 return False
151
152         def _rm_rf(self, path: Optional[str]) -> None:
```

```
153             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
154             if path:
155                 shutil.rmtree(path, ignore_errors=True)
156
157         def healthcheck(self) -> Tuple[bool, str]:
158             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
159             if not self.cfg.weights_path:
160                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
161             weights = os.path.expanduser(self.cfg.weights_path)
162             if not os.path.exists(weights):
163                 return False, f"WASB weights not found at {weights}"
164             repo_src = self._repo_src()
165             if not os.path.isdir(repo_src):
166                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
167                                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
168             # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
169             # argparse rc=2 from wasb_infer.py at run time.
170             if self.cfg.device not in VALID_DEVICES:
171                 return False, (f"unknown device {self.cfg.device!r} ? valid: {',
'.join(VALID_DEVICES)}.")
172             try:
173                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
174             except FileNotFoundError:
175                 return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
176             except subprocess.TimeoutExpired:
177                 return False, "native torch healthcheck timed out."
178             out = (res.stdout or "").strip()
179             if res.returncode != 0:
180                 return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
181             # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
182             self._cuda_available = "cuda True" in out
183             ctx = DeviceContext(self.cfg.device, self._cuda_available)
184             if ctx.cuda_required_but_missing:
185                 # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
186                 # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
187                 return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
188             if ctx.fell_back:
189                 logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
CPU; WASB "
190                                "inference will be slow. Set indexing.wasb.device=cuda to
require a GPU.", out)
191             return True, out
192
193         def run_predict(self, video_win_path: str, output_win_dir: str,
194                         video_id: Optional[str] = None) -> Optional[str]:
195             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
196
197             Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
198             runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
199             same downstream contract, so the parity comparison is apples-to-apples.
200             """
201             output_dir = os.path.abspath(output_win_dir)
202             os.makedirs(output_dir, exist_ok=True)
```

```
203             # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
204             norm = str(video_win_path).replace("\\", "/")
205             stem, _ext = os.path.splitext(os.path.basename(norm))
206             # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
207             key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
208             out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
209
210             if not self._copy_infer_script():
211                 logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
212                 return None
213
214             weights = os.path.expanduser(self.cfg.weights_path)
215             argv = [self.cfg.python_bin, "wasb_infer.py",
216                     "--video", os.path.abspath(str(video_win_path))]
217             frames_dir: Optional[str] = None
218             if self.cfg.stream_video:
219                 # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
220                 argv.append("--stream-video")
221             else:
222                 scratch = self.cfg.scratch_dir or output_dir
223                 frames_dir = os.path.join(scratch, f"{key}_frames")
224                 argv += ["--frames_out_dir", frames_dir]
225             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
226             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
227             device = self._effective_device()
228             argv += ["--weights", weights,
229                      "--sport", self.cfg.sport,
230                      "--device", device,
231                      "--out", out_csv]
232
233             logger.info("Running native WASB inference (device=%s%s) on %s ...",
234                         device, " [auto?fallback]" if self.cfg.device == AUTO and device !=
"cuda" else "",
235                         os.path.basename(norm))
236             try:
237                 res = self._run(argv, cwd=self._repo_src())
238             except FileNotFoundError:
239                 logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
240                 return None
241             except subprocess.TimeoutExpired:
242                 logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
243                 return None
244             if res.returncode != 0:
245                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
246                 return None
247             if not os.path.exists(out_csv):
248                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
249                 return None
250             logger.info("WASB trajectory CSV: %s", out_csv)
251
252             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
253             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
254             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
255             if not self.cfg.keep_frames and frames_dir:
```

```
256                 logger.info("Cache hygiene: removing frame cache for %s "
257                             "(set indexing.wasb.keep_frames=true to retain).", key)
258                 self._rm_rf(frames_dir)
259             return out_csv
260
261         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
262         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
263         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
264
```

14. user (2026-06-21T13:52:53.808Z)

```
Web search results for query: "ComfyUI Desktop Electron app installer auto installs python torch
GPU detection 2026"

Links: [{"title":"Manual Installation ? Local Self-Hosted -
ComfyUI","url":"https://docs.comfy.org/installation/manual_install"},{"title":"Install ComfyUI +
SageAttention 2.2 on Windows 10/11 | RTX 3000/4000/5000 Guide
2026","url":"https://smartart.live/articles/machine-learning/284-install-comfyui-
sageattention-22-on-windows-1011-ultimate-rtx-300040005000-guide-pytorch-cuda-triton-wheel-
picker-dll-fix-2026.html"},{"title":"ComfyUI Tutorial: SDXL & FLUX in 13 Steps
[2026]","url":"https://tech-insider.org/comfyui-tutorial-sdxl-flux-
workflow-13-steps-2026/"},{"title":"GitHub - Tavris1/ComfyUI-Easy-Install: Portable ComfyUI
installer for Windows, macOS and Linux with EZi Desktop app ? Nvidia GPU support ? Pixaroma
Community Edition","url":"https://github.com/Tavris1/ComfyUI-Easy-Install"},{"title":"GitHub -
Comfy-Org/desktop: The desktop app for ComfyUI (Windows & macOS) ·
GitHub","url":"https://github.com/Comfy-Org/desktop"},{"title":"How to install ComfyUI - Stable
Diffusion Art","url":"https://stable-diffusion-art.com/how-to-install-
comfyui/"},{"title":"ComfyUI Desktop Installation Guide | ComfyUI Wiki","url":"https://comfyui-
wiki.com/en/install/install-comfyui/comfyui-desktop-installation-guide"},{"title":"GitHub -
Comfy-Org/ComfyUI: The most powerful and modular diffusion model GUI, api and backend with a
graph/nodes interface. · GitHub","url":"https://github.com/comfy-org/comfyui"},{"title":"GitHub
- Rogala/ComfyUI-Toolkit: A set of Windows tools for installing, managing, updating, switching
versions, and running > ComfyUI + the PyTorch stack in a Python venv virtual environment for
NVIDIA GPUs. · GitHub","url":"https://github.com/Rogala/ComfyUI-Toolkit"},{"title":"Building
ComfyUI Desktop ? ComfyUI-nunchaku 1.2.1
documentation","url":"https://nunchaku.tech/docs/ComfyUI-
nunchaku/developer/build_comfyui_desktop.html"}]

Based on the search results, here's what I found about ComfyUI Desktop Electron app installer
regarding auto-installation of Python, PyTorch, and GPU detection in 2026:

## ComfyUI Desktop Overview

ComfyUI Desktop is the official desktop one-click installation version launched by ComfyUI,
which can be installed like regular software, supports quick installation, automatic Python
environment and dependency configuration, and supports importing existing ComfyUI settings,
models, workflows, and files.

## Automatic Installation Features

ComfyUI Desktop is an Electron-style native app that bundles Python, ComfyUI, and Manager into a
one-click installer.

Desktop uses its own bundled Python. This means the installer automatically includes Python
without requiring manual installation.

## GPU Support and Detection

For GPU capabilities, the Windows version requires NVIDIA graphics card (does not support AMD
GPU). The installer appears to handle GPU detection as part of its automatic dependency
```

configuration, though specific details about the automatic GPU detection mechanism aren't explicitly detailed in the search results.

Python and PyTorch Configuration

If the bundled ComfyUI version contains manager_requirements.txt, ComfyUI-Manager will be installed via pip at runtime; otherwise the legacy custom node is cloned for compatibility. This indicates that PyTorch and other dependencies are installed automatically during the installation process.

The installer also asks you to select a location to store ComfyUI files like models, inputs, outputs, custom_nodes and saved workflows, with this directory stored in the basePath key of config.json.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

15. user (2026-06-21T13:52:54.134Z)

Web search results for query: "uv pip torch index-url platform marker sys_platform conditional CUDA CPU pyproject 2026"

Links: [{"title":"Using uv with PyTorch | uv","url":"https://docs.astral.sh/uv/guides/integration/pytorch/"},{"title":"How to Install PyTorch with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-pytorch-with-uv/"},{"title":"dependencies - A Developer's Guide to Portable PyTorch Installation with uv and pip - portability","url":"https://iifx.dev/en/articles/460177044/a-developer-s-guide-to-portable-pytorch-installation-with-uv-and-pip"},{"title":"Managing PyTorch CPU/CUDA Versions per Environment with uv Part 2 | MJUN Tech Note","url":"https://mjunya.com/en/posts/2024-10-22-python-uv-pytorch-2/"},{"title":"Simplifying PyTorch Environment Setup with uv: A Unified Approach for Cross-Platform and CPU/GPU Support","url":"https://zenn.dev/haru256/articles/6ded722b409d13?locale=en"},{"title":"uv vs pixi vs conda for Scientific Python | pydevtools","url":"https://pydevtools.com/handbook/explanation/uv-vs-pixi-vs-conda-for-scientific-python/"},{"title":"Useful Features of the Python Project Management Tool uv up to v0.5.3 ? Dependencies Edition | by Turing Inc. | Medium","url":"https://medium.com/@turingmotors/5-useful-features-of-the-python-project-management-tool-uv-up-to-v0-5-3-dependencies-edition-9aab2cddceeb"},{"title":"pytorch setup with uv | mike bommarito","url":"https://michaelbommarito.com/wiki/programming/languages/python/pytorch-setup/"}]

Based on the search results, here's comprehensive information about using `uv`, `pip`, PyTorch, platform markers, and conditional CUDA/CPU installation with `pyproject.toml`:

Key Concepts

With `tool.uv.sources`, you can use environment markers to specify the desired index for each platform.

Platform-Specific Configuration with sys_platform

You can gate on `sys_platform` to instruct uv to use the PyTorch index on Linux and Windows, but fall back to PyPI on macOS, since PyTorch doesn't publish CUDA builds for macOS.

pyproject.toml Example

A configuration that uses PyTorch's CUDA-enabled builds on Linux and CPU-only builds on other platforms includes torch and torchvision dependencies with conditional markers like `sys_platform != 'linux'` for CPU versions and `sys_platform == 'linux'` for CUDA versions, along with [[tool.uv.index]] sections defining the PyTorch index URLs.

Conditional Markers

You can write torch dependencies for each environment using environment markers (PEP508) in the dependencies section, combining conditions like `sys_platform == 'linux' and platform_machine == 'x86_64'` for specific platform/architecture combinations.

The explicit Flag

The `explicit` flag indicates that the index should only be used for packages that explicitly specify it in `tool.uv.sources`. This prevents unrelated packages from being resolved from specialized indexes.

Quick Installation Methods

The uv pip interface offers a `--torch-backend` flag that selects the correct PyTorch index without any pyproject.toml configuration, and `--torch-backend=auto` makes uv detect the available GPU hardware and pick the appropriate backend.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

16. assistant (2026-06-21T13:52:58.656Z)

The uv `--torch-backend=auto` is a strong finding. Let me read the presets and startup checks to understand how DeviceContext ties to profiles, and fetch the authoritative uv PyTorch docs to confirm `--torch-backend=auto` details.

17. user (2026-06-21T13:52:59.633Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
```

```
    # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to warn
    # when config.json explicitly overrides compute_target into a state inconsistent with the profile.
    COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
        "truly-local": "local_gpu",
        "cloud-serving": "cloud_run",
        "cpu-mock": "cpu_mock",
    }

    # A preset is a partial, nested override dict over the model defaults. It touches ONLY knobs
    # that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute_target}`` plus
    # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones extend
    # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
    # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
    PROFILE_PRESETS: dict[str, dict[str, Any]] = {
        "truly-local": {
            "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
            # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
            # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
            "indexing": {"skip_ai_handoff": True},
            "storage": {"backend": "local"},
        },
        "cloud-serving": {
            "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
            # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11).
            "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
            "storage": {"backend": "gcs"},
        },
        "cpu-mock": {
            "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
            # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
            "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
            "storage": {"backend": "local"},
        },
    }


    def is_known_profile(profile: str) -> bool:
        """True for a non-empty, recognised profile name."""
        return bool(profile) and profile in PROFILE_PRESETS


    def preset_for(profile: str) -> dict[str, Any]:
        """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
        module-level table), or ``{}`` for ``""`` / an unknown profile."""
        return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))


    def _truthy(val: Optional[str]) -> bool:
        """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
        if val is None:
            return False
        return val.strip().lower() in {"1", "true", "yes", "on"}


    def _cuda_available() -> bool:
        """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
```

```
and
86          ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87          try:
88              # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89              # and optional). Only reached when a caller opts into probe_cuda.
90              import torch
91
92              return bool(torch.cuda.is_available())
93          except Exception:
94              return False
95
96
97      def probe_cuda_available() -> bool:
98          """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101         that *wants* the GPU heads-up can pass the result in without the startup module
importing
102         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105         error that could false-fail."""
106         return _cuda_available()
107
108
109     def suggest_profile(
110         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111     ) -> str:
112         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114         Signals, in priority order:
115
116           - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
117           - ``CI`` truthy                        ? ``"cpu-mock"``
118           - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119           - otherwise                            ? ``""`` (no strong suggestion)
120
121         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
123         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
124         """
125         e = os.environ if env is None else env
126         if e.get("K_SERVICE"):
127             return "cloud-serving"
128         if _truthy(e.get("CI")):
129             return "cpu-mock"
130         if probe_cuda and _cuda_available():
131             return "truly-local"
132         return ""
133
```

18. user (2026-06-21T13:53:07.499Z)

PyTorch Installation Across Platforms with uv

(1) Automatic Backend Detection

The `--torch-backend=auto` feature automatically selects the appropriate PyTorch index. According to the documentation, when enabled, "uv will query for the installed CUDA driver, AMD GPU versions, and Intel GPU presence, then use the most-compatible PyTorch index for all relevant packages." If no GPU is detected, it falls back to CPU-only builds.

Usage:
```bash
uv pip install torch --torch-backend=auto
```
or
```
UV_TORCH_BACKEND=auto uv pip install torch
```

Important caveat: "At present, `--torch-backend` is only available in the `uv pip` interface" ? not in the project-based workflow.

(2) Platform-Specific Index Configuration

Using `tool.uv.sources` with environment markers pins different indices per platform. For example, to use CUDA 13.0 on Linux and CPU-only elsewhere:

```toml
[tool.uv.sources]
torch = [
  { index = "pytorch-cpu", marker = "sys_platform != 'linux'" },
  { index = "pytorch-cu130", marker = "sys_platform == 'linux'" },
]

[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true

[[tool.uv.index]]
name = "pytorch-cu130"
url = "https://download.pytorch.org/whl/cu130"
explicit = true
```

(3) macOS/Apple Silicon Limitations

The documentation notes: "PyTorch doesn't publish CUDA builds for macOS." The guide shows configurations that gate CUDA indices to Linux/Windows only, falling back to PyPI on macOS. However, **the guide does not mention Apple MPS (Metal Performance Shaders) support.**

(4) Version Requirements

"Some of the features outlined in this guide require uv version 0.5.3 or later. We recommend upgrading prior to configuring PyTorch."

19. assistant (2026-06-21T13:53:11.841Z)

I now have strong material. Let me read the startup.py per-profile checks and check the worker/main entrypoints briefly for how device config flows, plus confirm the WASB two-env / weights-from-GCS facts which are load-bearing for "fit_for_this_app."

20. user (2026-06-21T13:53:12.344Z)

```
1       """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3       A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a
```

```
coherent
4       runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
5       (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7         * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one
config bug we
8           can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9           bucket can never serve). This is the README §4.3 "loud error before any job".
10        * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
11          falling back to CPU) ? a **warning**, never an error: the probe is best-effort
(torch may live
12          only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the
detector's
13          own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14        * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on
Cloud Run) ?
15          a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
16          we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18      **DEFAULT-OFF:** with no profile selected (``profile == ""``) this is a strict **no-op**
? exactly
19      the pre-M5 behaviour. Only an explicitly-selected profile triggers any check (mirrors
the loader,
20      which applies a preset only for an explicit profile). So nothing changes for today's
manual-knob
21      deployments.
22      """
23
24      from __future__ import annotations
25
26      import logging
27      import os
28      from dataclasses import dataclass
29      from typing import Any, Mapping, Optional
30
31      LEVEL_ERROR = "error"
32      LEVEL_WARN = "warn"
33
34      # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
35      _CLOUD_STORAGE = ("gcs", "s3")
36      # STRONG env signals that Google ADC *might* resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /
37      # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
config dir
38      # with no active login), so they would suppress the heads-up falsely. (s3 creds are
intentionally
39      # left to boto3's default chain ? no parallel heads-up, by design.)
40      _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")
41
42
43      @dataclass(frozen=True)
44      class StartupCheck:
45          """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
up)."""
46
47          level: str
48          code: str       # stable, greppable code (e.g. "STORAGE_INCOHERENT")
49          message: str
50
51
```

```
    class StartupCheckError(RuntimeError):
        """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
check."""

        def __init__(self, failures: list[StartupCheck]):
            self.failures = failures
            super().__init__(
                "deployment profile startup check failed: "
                + "; ".join(f"[{c.code}] {c.message}" for c in failures)
            )


    def _creds_discoverable(env: Mapping[str, str]) -> bool:
        """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
False here
        only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""
        return any(env.get(k) for k in _GCP_ENV_HINTS)


    def run_startup_checks(
        config: Any,
        *,
        cuda_available: Optional[bool] = None,
        env: Optional[Mapping[str, str]] = None,
    ) -> list[StartupCheck]:
        """Return the per-profile findings for ``config`` (empty when no profile is
selected).

        ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
GPU checks
        are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available()``).
        Pure: no IO, no torch import ? the caller owns the probe so this stays importable
everywhere.
        """
        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        if not profile:
            return []  # DEFAULT-OFF: no profile ? no checks (identical to pre-M5).

        e = os.environ if env is None else env
        storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
        indexing = getattr(config, "indexing", None)
        detector_impl = getattr(indexing, "detector_impl", "auto")
        device = getattr(getattr(indexing, "wasb", None), "device", "cuda")

        checks: list[StartupCheck] = []

        if profile == "truly-local":
            # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
            # an inconsistent (but not fatal) choice worth surfacing.
            if storage_backend not in ("local", "gdrive"):
                checks.append(StartupCheck(
                    LEVEL_WARN, "STORAGE_UNUSUAL",
                    f"profile 'truly-local' usually uses local/gdrive storage, but
storage.backend="
                    f"'{storage_backend}'. Reads will go to a remote backend.",
                ))
            # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
            # best-effort and may be False even when a subprocess detector env has CUDA).
            if cuda_available is False:
                if device == "cuda":
                    checks.append(StartupCheck(
```

```
                        LEVEL_WARN, "CUDA_NOT_VISIBLE",
                        "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
                        "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
                        "'auto' for the CPU fallback (M4).",
                    ))
                elif device == "auto":
                    checks.append(StartupCheck(
                        LEVEL_WARN, "CPU_FALLBACK",
                        "device='auto' and no CUDA GPU is visible to THIS process ? auto may
resolve "
                        "to CPU (much slower). If torch lives only in the detector
subprocess env this "
                        "can be a false alarm; the runner's DeviceContext/healthcheck is the
authority.",
                    ))

        elif profile == "cloud-serving":
            # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
            if storage_backend not in _CLOUD_STORAGE:
                checks.append(StartupCheck(
                    LEVEL_ERROR, "STORAGE_INCOHERENT",
                    f"profile 'cloud-serving' requires a cloud storage backend "
                    f"({'/'.join(_CLOUD_STORAGE)}), but storage.backend='{storage_backend}'.
"
                    "A cloud job cannot read/write a local-only backend.",
                ))
            elif storage_backend == "gcs" and not _creds_discoverable(e):
                checks.append(StartupCheck(
                    LEVEL_WARN, "CREDS_UNCONFIRMED",
                    "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "
                    "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
                ))
            if device == "cpu":
                checks.append(StartupCheck(
                    LEVEL_WARN, "CLOUD_CPU_DEVICE",
                    "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
                    "detection will not use the GPU.",
                ))
            elif cuda_available is False and device in ("cuda", "auto"):
                checks.append(StartupCheck(
                    LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
                    "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
                ))

        elif profile == "cpu-mock":
            # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
            if detector_impl != "stub":
                checks.append(StartupCheck(
                    LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
                    f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
                    f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
                ))

        return checks


```

```
159     def enforce_startup_checks(
160         config: Any,
161         *,
162         cuda_available: Optional[bool] = None,
163         env: Optional[Mapping[str, str]] = None,
164         raise_on_error: bool = True,
165         logger: Optional[logging.Logger] = None,
166     ) -> list[StartupCheck]:
167         """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
168         ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
169
170         Returns the full findings list (so a caller can inspect/surface them). A no-op with
no logging
171         and no raise when no profile is selected (``run_startup_checks`` returns ``[]``)."""
172         log = logger or logging.getLogger(__name__)
173         checks = run_startup_checks(config, cuda_available=cuda_available, env=env)
174         for c in checks:
175             if c.level == LEVEL_ERROR:
176                 log.error("[STARTUP] %s: %s", c.code, c.message)
177             else:
178                 log.warning("[STARTUP] %s: %s", c.code, c.message)
179         errors = [c for c in checks if c.level == LEVEL_ERROR]
180         if errors and raise_on_error:
181             raise StartupCheckError(errors)
182         return checks
183
```

21. user (2026-06-21T13:53:13.618Z)

```
71:    # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72:    # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4 CPU
fallback
73:    # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-CPU
downgrade).
74:    device: str = "cuda"        # auto | cuda | mps | cpu
```

22. assistant (2026-06-21T13:53:18.120Z)

I have a thorough grounding. Let me do two more targeted searches to round out the picture: how Whisper-family desktop apps (whisper.cpp / WhisperX / Buzz) ship cross-platform, and the modern "installer/launcher picks the wheel" pattern (Stability Matrix / pinokio / one-click launchers).

23. user (2026-06-21T13:53:25.418Z)

Web search results for query: "PyInstaller bundle torch CUDA huge size CPU wheel desktop AI app distribution problem"

Links: [{"title":"Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion #8552","url":"https://github.com/orgs/pyinstaller/discussions/8552"},{"title":"How to reduce the size of the PyTorch package after using PyInstaller with -D option, resulting in a 3GB size for the torch file? - PyTorch Forums","url":"https://discuss.pytorch.org/t/how-to-reduce-the-size-of-the-pytorch-package-after-using-pyinstaller-with-d-option-resulting-in-a-3gb-size-for-the-torch-file/194822"},{"title":"PyInstaller, issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/7175"},{"title":"Reducing the Size of PyTorch-Based Executable · Issue #8551 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/8551"},{"title":"Has anyone PyInstaller working for Spacy with CUDA? · explosion/spaCy · Discussion #10153","url":"https://github.com/explosion/spaCy/discussions/10153"},{"title":"PyTorch 2.0 distribution that uses cuda only if available? - PyTorch Forums","url":"https://discuss.pytorch.org/t/pytorch-2-0-distribution-that-uses-cuda-only-if-available/177230"}]

Based on the search results, here's what I found about the PyInstaller bundle size problem with

PyTorch and CUDA:

The Core Problem

Most of the size in PyTorch bundles are binary extensions and CUDA/cuDNN shared libraries, which cannot be split. This is a fundamental challenge when distributing AI applications.

Size Reality

PyTorch 2.3 with CUDA 11.8 (torch2.3+cu118) in conda is around 5GB, and the torch library in PyInstaller apps is about 4GB. For comparison, PyTorch 2.0 without CUDA is ~620MB, while the version bundled with CUDA 11.7 is 1.8GB.

Key Insight

Using torch while keeping executable files small are mutually exclusive aspects, especially for CUDA-enabled torch builds. Smaller executables (200-300 MB) typically use either CPU-only torch builds (no CUDA) or older builds with smaller CUDA libraries.

Practical Considerations

Some developers give up on the `--onefile` option because building takes forever and so does the first invocation, instead using `--onedir` and then zipping for convenience.

The bottom line is that CUDA-enabled PyTorch distributions are inherently large due to the libraries required for GPU computation, making desktop distribution challenging without accepting the large file size or compromising on GPU support.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

24. user (2026-06-21T13:53:27.148Z)

Web search results for query: "Stability Matrix pinokio one-click launcher detects GPU installs correct torch wheel python venv"

Links: [{"title":"Pinokio: A One-Click Playground for Running AI Models Locally","url":"https://diffusiondoodles.substack.com/p/pinokio-a-one-click-playground-for"},{"title":"Pinokio","url":"https://beta.pinokio.co/"},{"title":"GitHub - timoncool/VoxCPM2_portable-pinokio: One-click Pinokio launcher for VoxCPM2 Portable. Multilingual TTS (30 languages), Voice Design, Voice Cloning, end-to-end LoRA fine-tuning. Cross-platform (Win/Linux/macOS, NVIDIA/AMD/CPU). · GitHub","url":"https://github.com/timoncool/VoxCPM2_portable-pinokio"},{"title":"Stability Matrix ? Grokipedia","url":"https://grokipedia.com/page/Stability_Matrix"},{"title":"GitHub - PierrunoYT/dots.tts-Pinokio · GitHub","url":"https://github.com/PierrunoYT/dots.tts-Pinokio"},{"title":"Problem launching ComfyUI with Stability Matrix after updates · Issue #1510 · LykosAI/StabilityMatrix","url":"https://github.com/LykosAI/StabilityMatrix/issues/1510"}]

Based on the search results, here's what I found about Pinokio and Stability Matrix:

Pinokio One-Click Launcher

Pinokio functions like a smart script manager with a graphical interface that can launch apps, scripts, and environments with a single click. Its strength lies in reducing the friction involved in setting up complex tooling, especially for AI models that often have multiple dependencies, environment variables, and GPU requirements.

GPU and PyTorch Detection

When you click Install inside Pinokio, it will clone the app, create a Python 3.12 venv, and install PyTorch with the right build for your GPU. More specifically, PyTorch is auto-selected

by GPU/OS ? CUDA 12.8 (NVIDIA x64), CUDA 13.0 (aarch64), DirectML (AMD Win), ROCm 6.3 (AMD Linux), MPS/CPU (macOS), with CPU fallback.

Another example shows that Pinokio auto-configures CUDA for Nvidia GPUs and Vulkan for AMD/Intel Arc cards.

Stability Matrix

In contrast, a workaround in Stability Matrix involves installing a compatible PyTorch build by navigating to Packages > ComfyUI > 3-dot menu > Python Packages and adding/installing the command with the correct wheel URL. Additionally, Stability Matrix lacks a built-in "Open Terminal" feature that automatically activates the virtual environment (venv) for ComfyUI.

Pinokio appears to automate the GPU detection and PyTorch installation process more seamlessly than Stability Matrix requires.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

25. assistant (2026-06-21T13:53:36.932Z)

I have comprehensive grounding across both our code and the comparable apps. I have enough to synthesize the research output now.

Key code facts confirmed:
- `backend/pipeline/detectors/device.py:32-56` ? `DeviceContext(requested, cuda_available)` with `effective`/`fell_back`/`cuda_required_but_missing`; `auto` = cuda-if-available-else-cpu, explicit `cuda` stays strict, `cpu`/`mps` verbatim.
- `pyproject.toml:8-11,58-65` ? torch/torchvision deliberately NOT in deps; `gpu` extra is empty/docs-only with both `--index-url` recipes commented.
- `native_wasb_runner.py:56,116-136` ? separate `wasb` env python invoked by path, subprocess `torch.cuda.is_available()` probe (`_CUDA_PROBE_CODE`), env-first config (`WASB_DEVICE`/`WASB_PYTHON`/`WASB_WEIGHTS`).
- `presets.py:84-106` ? lazy/best-effort torch CUDA probe, no torch at module load.
- `startup.py:104-145` ? GPU-mismatch is a WARNING not error (probe is best-effort; subprocess env is the authority).

Now calling StructuredOutput.

26. user (2026-06-21T13:54:56.500Z)

Structured output provided successfully

27. assistant (2026-06-21T13:55:07.596Z)

Research complete. Returned via StructuredOutput.

Summary of what I grounded and found:

Our run-time device handling is already mature and mirrors the comparable apps. `backend/pipeline/detectors/device.py:32-56` (`DeviceContext`) encodes exactly the auto/cuda/cpu/mps contract Ollama and faster-whisper use at run time, and `backend/config/startup.py:104-145` already emits the loud "GPU not detected, running slow" warnings (CPU_FALLBACK / CUDA_NOT_VISIBLE) that Ollama notably lacks.

The actual gap is install-time wheel selection, which `pyproject.toml:8-11,58-65` correctly refuses to express (torch out of deps, empty docs-only `gpu` extra). The five options I mapped: (A) uv `--torch-backend=auto`, (B) static tool.uv.sources marker pinning, (C) installer/launcher that probes-and-picks (Pinokio / ComfyUI-Desktop model), (D) CPU-default + documented GPU upgrade, (E) PyInstaller single bundle.

Recommendation: a first-run launcher (C) built on uv `--torch-backend=auto` (A) into the bundled venv ? detection then agrees with `DeviceContext('auto')` by construction. Keep pyproject as-is.

Drive the WASB torch-1.11 conda env as a SEPARATE bootstrap step (it cannot share the main torch-2.6 env), pulling weights from GCS in the same step. Avoid E (4-5GB CUDA bundle can't serve CPU+GPU from one artifact, and the two-torch-env reality makes a frozen image intractable).

Relevant files:
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\device.py
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\presets.py
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\startup.py
- C:\Users\avidu\Projects\badminton-highlight-indexer\pyproject.toml